HDFS + Spark + MLlib

Ingeniería para el procesado de datos masivos | Actividad 1

Fecha límite de entrega: 15 de junio 2026

Estudiante: César Goavanni Gómez Camargo

Contenido

1. Levantamiento de servicios Docker 2
2. Verificación de servicios para confirmar funcionamiento HDFS 3
3. Creación de carpeta 4
4. Carga de csv a HDFS 4
5. Inicializando y cargando dataset a Spark 6
   Carga del dataset de HDFS a Spark 8
6. Utilizando Spark SQL y MLlib 9
7. Mejoras que se pueden aplicar 10
Referencias 10

1. Levantamiento de servicios Docker

En otros proyectos utilizaba un Dockerfile para configurar la versión y las dependencias del entorno Python que iba a utilizar y las definía en el archivo requirements.txt; para esta actividad, el servicio spark-jupyter se levanta con la imagen oficial jupyter/pyspark-notebook que es un entorno todo en uno mantenido por la comunidad de Jupyter, la cual incluye de forma nativa:

1. Una instalación completa de Python.
2. Todo el entorno de Jupyter Notebook / JupyterLab listo para usar.
3. El motor de Apache Spark y la librería PySpark preinstalados y configurados para entenderse con la versión de Python que ya incluye.

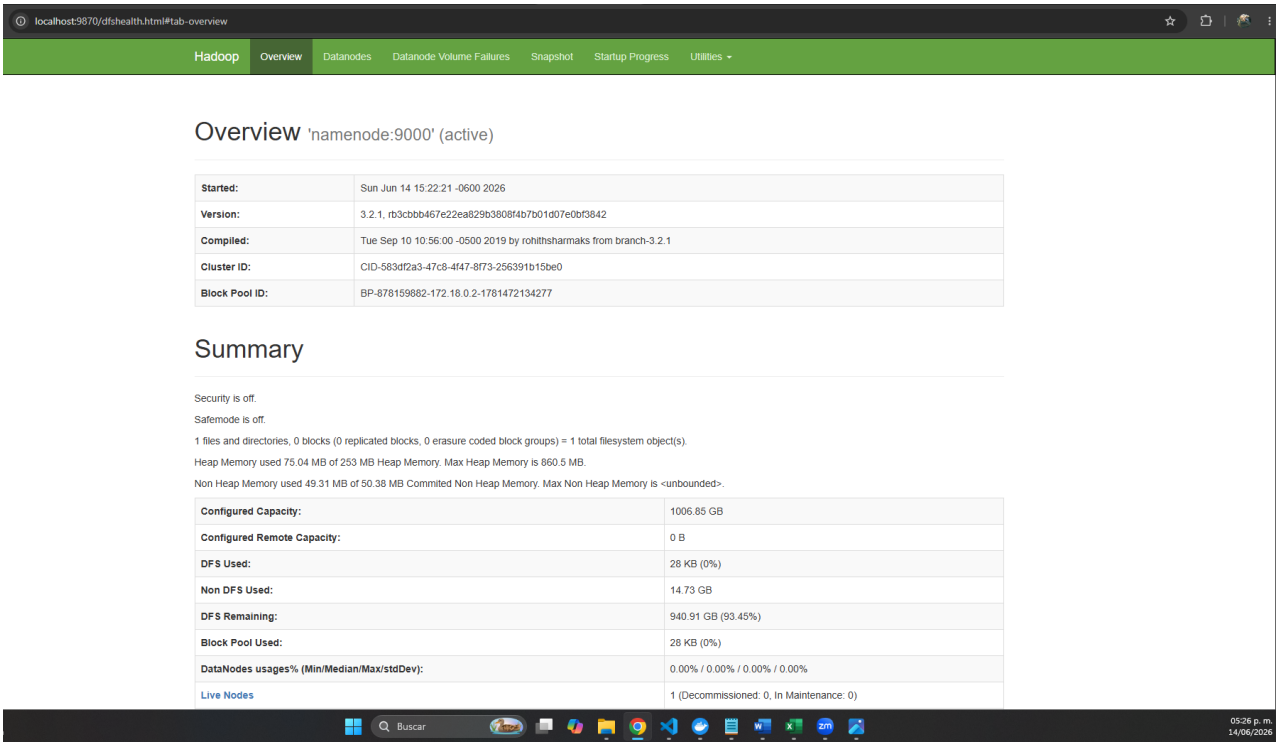
Toda vez que el desarrollo de esta actividad utiliza el motor Apache Spark, el archivo docker-compose.yml no requiere el servicio de ResourceManager ni tampoco del NodeManager para realizar trabajos de MapReduce, en su lugar Spark gestiona sus propios recursos para interactuar con HDFS sin necesidad de un clúster YARN completo desplegado en Docker, esto nos ahora un gran número de recursos de RAM y CPU en el equipo personal.

```
PS C:\Users\Cesar\Documents\Financial_Fraud> docker-compose up -d
time="2026-06-14T15:05:24-06:00" level=warning msg="C:\\Users\\Cesar\\Documents\\Financial_Fraud\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, pl
ease remove it to avoid potential confusion"
[+] up 47/55
 ✓ Image bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8 Pulled                                                                                                    856.8s
 ✓ Image bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8 Pulled                                                                                                    856.8s
 ✓ Image jupyter/pyspark-notebook                        Pulled                                                                                                    989.4s
 ✓ Network financial_fraud_bigdata                       Created                                                                                                     0.6s
 ✓ Volume financial_fraud_dataset                        Created                                                                                                     0.1s
 ✓ Container namenode                                    Started                                                                                                     8.5s
 ✓ Container datanode                                    Started                                                                                                     7.9s
 ✓ Container spark-jupyter                               Started                                                                                                     8.6s
PS C:\Users\Cesar\Documents\Financial_Fraud> docker ps
CONTAINER ID   IMAGE                                          COMMAND                  CREATED       STATUS                   PORTS
                                NAMES
be7cca292361   jupyter/pyspark-notebook                       "tini -g -- start-no…"   2 hours ago   Up 2 hours (healthy)     0.0.0.0:8888->8888/tcp, [::]:8888->8888/tcp
                                spark-jupyter
18a4446715d0   bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8   "/entrypoint.sh /run…"   2 hours ago   Up 2 hours (healthy)     0.0.0.0:9864->9864/tcp, [::]:9864->9864/tcp
                                datanode
1b5c06c10523   bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8   "/entrypoint.sh /run…"   2 hours ago   Up 2 hours (healthy)     0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp, 0.0.0
.0:9870->9870/tcp, [::]:9870->9870/tcp   namenode
PS C:\Users\Cesar\Documents\Financial_Fraud>
```

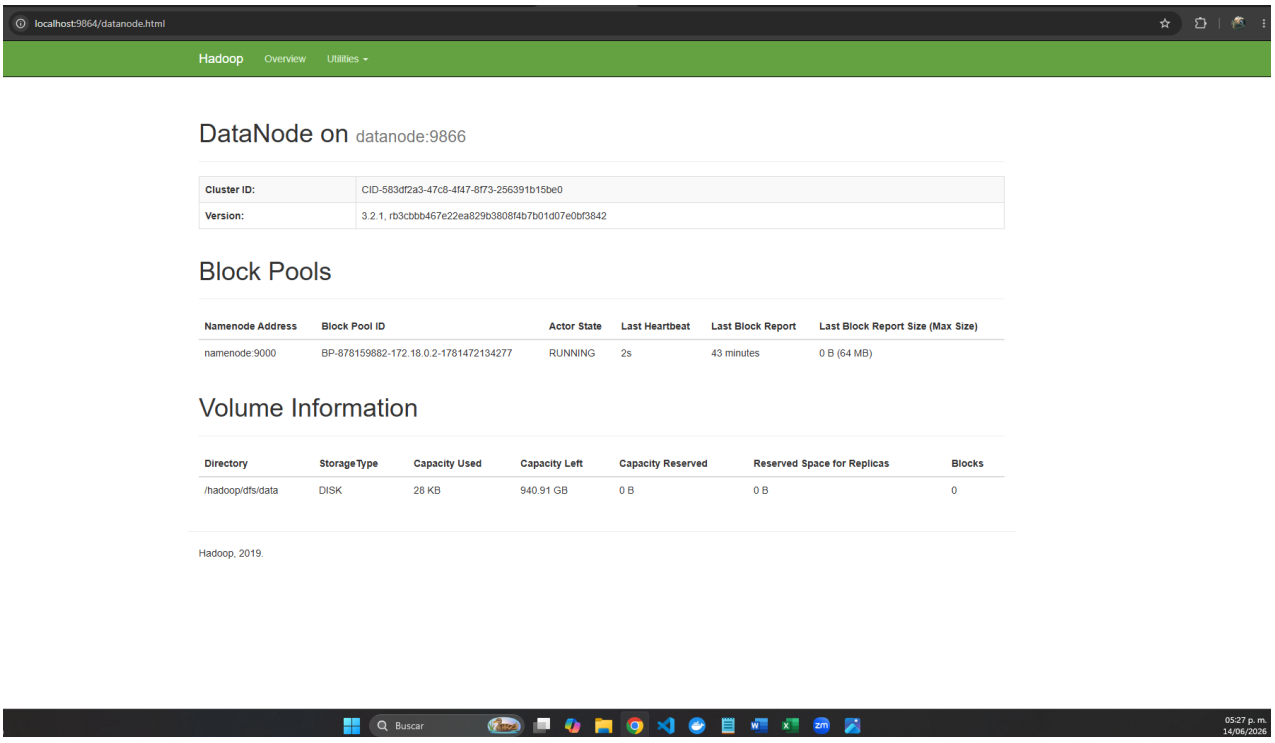
El levantamiento de los servicios se demoró aproximadamente 16 minutos en descargar las imágenes al ser la primera ocasión en que lo hacía y en iniciar los servicios menos de 10 segundos, cabe mencionar que se creó un volumen nombrado “dataset” para mantener la persistencia de la base de datos objetivo de este análisis.

2. Verificación de servicios para confirmar funcionamiento HDFS

Al ingresar a la url http://localhost:9870 se despliega la información del namenode.



Al ingresar a la url http://localhost:9864 se despliega la información del datanode.



3. Creación de carpeta

Dado que mi sistema operativo es Windows, ejecuté en una sola línea el comando para crear la carpeta ya que el símbolo \ no es reconocido como salto de línea.

```
PS C:\Users\Cesar\Documents\Financial_Fraud> docker exec -it namenode hdfs dfs -mkdir -p /cesar

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug namenode
    Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Cesar\Documents\Financial_Fraud> docker exec -it namenode hdfs dfs -ls /cesar

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug namenode
    Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Cesar\Documents\Financial_Fraud>
```

```
PS C:\Users\Cesar\Documents\Financial_Fraud> docker exec -it namenode hdfs dfs -ls /
Found 1 items
drwxr-xr-x   - root supergroup          0 2026-06-14 23:54 /cesar
```

4. Carga de csv a HDFS

Con la intensión de incrementar la eficiencia al trabajar con el dataset https://www.kaggle.com/datasets/umitka/synthetic-financial-fraud-dataset configuré un volumen de montaje llamado datasets, por lo que la copia del archivo .csv al contenedor en la carpeta tmp ya no fue necesario mediante el comando:

```
docker cp synthetic_fraud_dataset_trunc.csv namenode:/tmp/synthetic_fraud_dataset_trunc.csv
```

Con base en lo anterior, procedí a subir el archivo .csv a HDFS con el siguiente comando ya que el volumen lo configuré en el datanode:

```
docker exec -it datanode hdfs dfs -put /datasets/synthetic_fraud_dataset_trunc.csv /cesar/
```

```
PS C:\Users\Cesar\Documents\Financial_Fraud> docker exec -it datanode hdfs dfs -put /datasets/synthetic_fraud_dataset_trunc.csv /cesar/
2026-06-15 00:40:52,575 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrusted = false

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug datanode
    Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Cesar\Documents\Financial_Fraud>
```

Confirmé que se cargó con el siguiente comando

```
PS C:\Users\Cesar\Documents\Financial_Fraud> docker exec -it namenode hdfs dfs -ls /cesar
Found 1 items
-rw-r--r--   1 root supergroup     131173 2026-06-15 00:40 /cesar/synthetic_fraud_dataset_trunc.csv

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug namenode
    Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Cesar\Documents\Financial_Fraud>
```

Y revisé el contenido con el comando:

`docker exec -it namenode hdfs dfs -cat /cesar/synthetic_fraud_dataset_trunc.csv`

```
2957,915,126.4585183,Online,Electronics,UK,7,0.036629466,0.187628448,0
7667,500,180.8913362,Online,Travel,UK,17,0.232427068,0.088262011,0
4403,192,101.2813836,POS,Food,DE,15,0.246494038,0.040760455,0
4998,421,84.21209959,ATM,Travel,TR,7,0.082153745,0.282143207,0
7774,511,143.2479103,QR,Grocery,US,20,0.16164735,0.289248729,0
2582,794,96.95341824,ATM,Clothing,DE,12,0.063156643,0.077961832,0
1590,499,110.5782417,ATM,Clothing,US,15,0.245147545,0.020661081,0
662,57,57.36574919,POS,Travel,UK,13,0.277339491,0.191470458,0
5364,419,94.78728423,ATM,Clothing,TR,22,0.238252551,0.294272034,0
```

Con esto confirmamos que el archivo .csv se encuentra en el sistema de archivos distribuidos de Hadoop en la ruta /cesar y pude conocer la información del archivo en HDFS.

```
PS C:\Users\Cesar\Documents\Financial_Fraud> docker exec -it namenode hdfs fsck /cesar/synthetic_fraud_dataset_trunc.csv -files -blocks -locations
Connecting to namenode via http://namenode:9870/fsck?ugi=root&files=1&blocks=1&locations=1&path=%2Fcesar%2Fsynthetic_fraud_dataset_trunc.csv
FSCK started by root (auth:SIMPLE) from /172.18.0.2 for path /cesar/synthetic_fraud_dataset_trunc.csv at Mon Jun 15 00:52:04 UTC 2026
/cesar/synthetic_fraud_dataset_trunc.csv 131173 bytes, replicated: replication=1, 1 block(s):  OK
0. BP-646772610-172.18.0.2-1781483520710:blk_1073741825_1001 len=131173 Live_repl=1  [DatanodeInfoWithStorage[172.18.0.3:9866,DS-3688b7c5-e885-4a4b-8621-e6207ad8668d,DISK]]


Status: HEALTHY
 Number of data-nodes:  1
 Number of racks:               1
 Total dirs:                    0
 Total symlinks:                0

Replicated Blocks:
 Total size:    131173 B
 Total files:   1
 Total blocks (validated):      1 (avg. block size 131173 B)
 Minimally replicated blocks:   1 (100.0 %)
 Over-replicated blocks:        0 (0.0 %)
 Under-replicated blocks:       0 (0.0 %)
 Mis-replicated blocks:         0 (0.0 %)
 Default replication factor:    1
 Average block replication:     1.0
 Missing blocks:                0
 Corrupt blocks:                0
 Missing replicas:              0 (0.0 %)

Erasure Coded Block Groups:
 Total size:    0 B
 Total files:   0
 Total block groups (validated):        0
 Minimally erasure-coded block groups:  0
 Over-erasure-coded block groups:       0
 Under-erasure-coded block groups:      0
 Unsatisfactory placement block groups: 0
 Average block group size:      0.0
 Missing block groups:          0
 Corrupt block groups:          0
 Missing internal blocks:       0
FSCK ended at Mon Jun 15 00:52:04 UTC 2026 in 50 milliseconds


The filesystem under path '/cesar/synthetic_fraud_dataset_trunc.csv' is HEALTHY

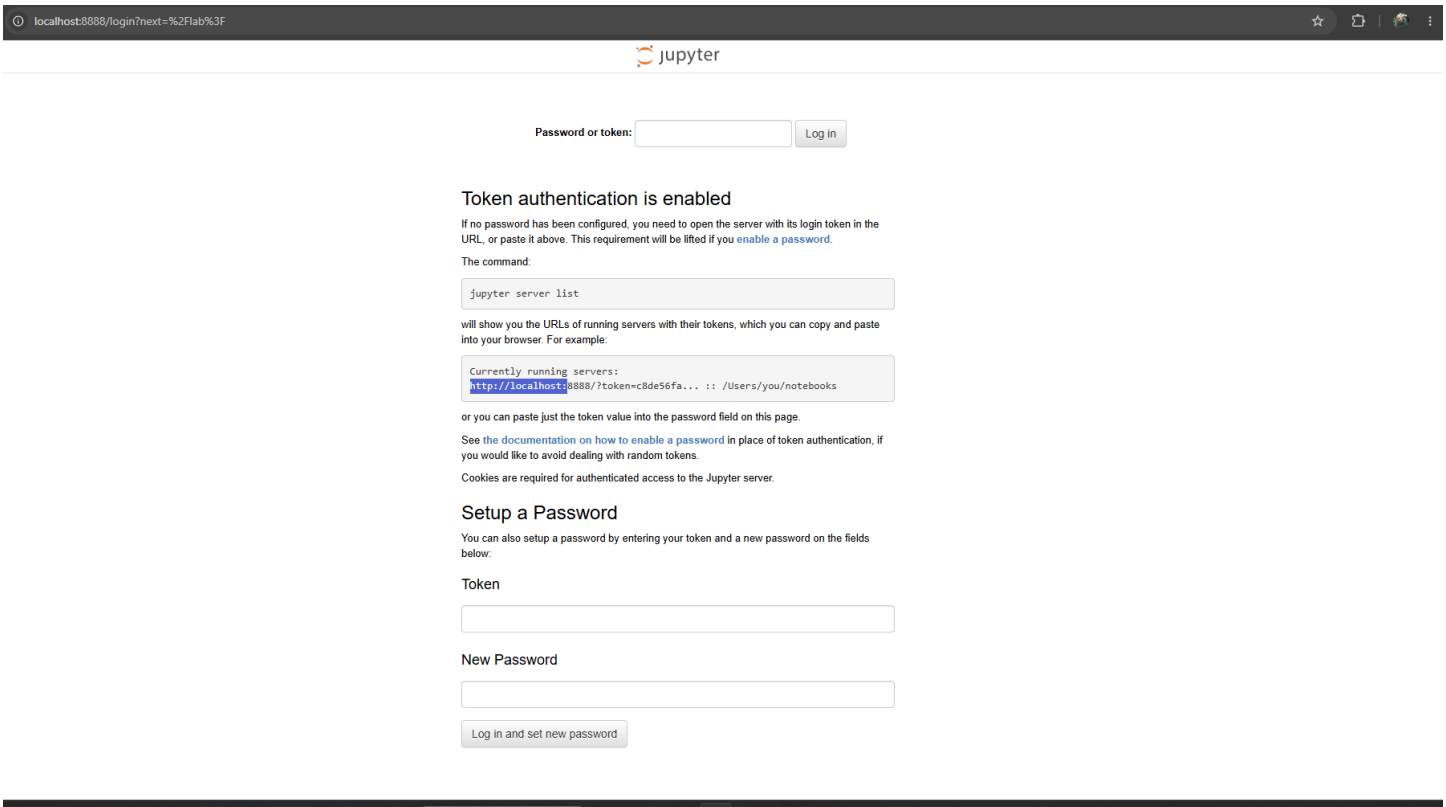
What's next:
```

5. Inicializando y cargando dataset a Spark

Ejecuté el siguiente comando para inicializar el servicio.

```
PS C:\Users\Cesar\Documents\Financial_Fraud> docker start spark-jupyter
spark-jupyter
PS C:\Users\Cesar\Documents\Financial_Fraud>
```

Al ingresar a la url http://localhost:8888 para ingresar al servicio spark-jupyter, se pide un token para ingresar el cual fue creado automáticamente al momento de levantar el servicio, por lo que fue necesario ejecutar el comando Docker logs spark-jupyter para conocerlo y colocarlo en la aplicación para contar con el acceso. Existe otra forma en la que podemos asignar un token desde el archivo .yml colocando en la sección de environment – JUPYTER_TOKEN=contraseña_a_asignar y luego realizar el levantamiento del servicio.

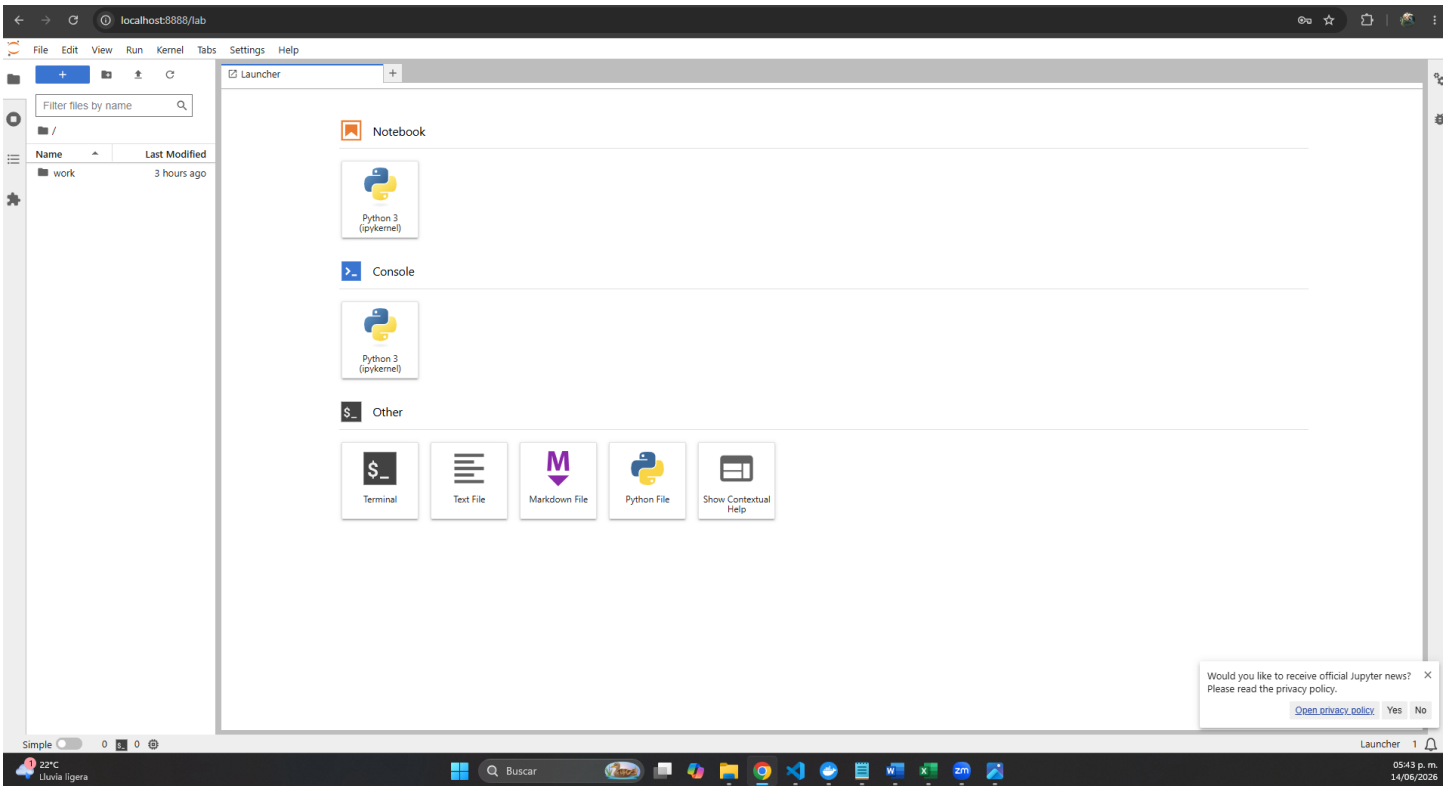





```
[I 2026-06-14 21:22:18.451 ServerApp] http://spark-jupyter:8888/lab?token=1c4463eed44d3af22272c44999329592e06bf4c1cc3568d6
[I 2026-06-14 21:22:18.451 ServerApp]     http://127.0.0.1:8888/lab?token=1c4463eed44d3af22272c44999329592e06bf4c1cc3568d6
[I 2026-06-14 21:22:18.452 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 2026-06-14 21:22:18.465 ServerApp]

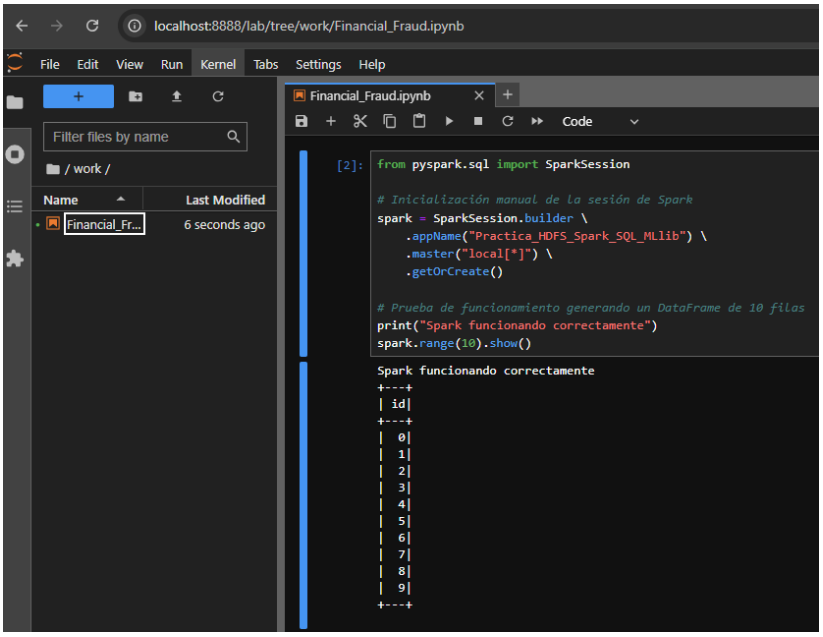
    To access the server, open this file in a browser:
        file:///home/jovyan/.local/share/jupyter/runtime/jpserver-6-open.html
    Or copy and paste one of these URLs:
        http://spark-jupyter:8888/lab?token=1c4463eed44d3af22272c44999329592e06bf4c1cc3568d6
        http://127.0.0.1:8888/lab?token=1c4463eed44d3af22272c44999329592e06bf4c1cc3568d6
[I 2026-06-14 21:22:22.798 ServerApp] Skipped non-installed server(s): bash-language-server, dockerfile-language-server-nodejs, javascript-typescript-langserver, jedi-language-server, julia-language-server, pyright, python-language-server, python-lsp-server, r-languageserver, sql-language-server, texlab, typescript-language-server, unified-language-server, vscode-css-languageserver-bin, vscode-html-languageserver-bin, vscode-json-languageserver-bin, yaml-language-server
[I 2026-06-14 23:27:39.237 ServerApp] 302 GET / (@172.18.0.1) 24.92ms
[I 2026-06-14 23:27:39.397 LabApp] 302 GET /lab? (@172.18.0.1) 72.46ms
PS C:\Users\Cesar\Documents\Financial_Fraud>
```

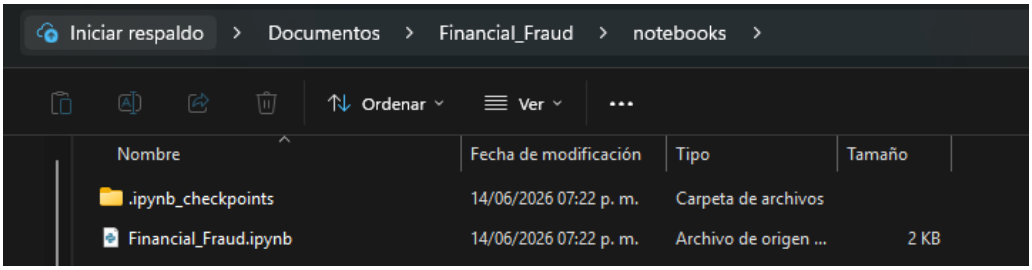
El token es la cadena que sigue después de http://127.0.0.1:8888/lab?token=



Esta es la pantalla de ingreso a la aplicación spark-jupyter.

Realicé la creación de Python 3 (ipykernel) en Notebook e inicié sesión en PySpark confirmando el correcto funcionamiento al mostrar un dataframe de 10 filas. Renombré el notebook y lo guardé en la carpeta work que está con el volumen de montaje notebooks.





Se confirma que el notebook también está en máquina local. Cabe mencionar que hay un total de 2,000 registros y que se puede encontrar la totalidad del dataset en https://www.kaggle.com/datasets/umitka/synthetic-financial-fraud-dataset

Carga del dataset de HDFS a Spark

```
Carga de csv desde HDFS
```

```
[3]: df = spark.read.csv(
        "hdfs://namenode:9000/cesar/synthetic_fraud_dataset_trunc.csv",
        header=True,
        inferSchema=True
    )
    df.show()
    df.printSchema()
```

```
+--------------+-------+-----------+----------------+-----------------+-------+----+-----------------+-------------+--------+
|transaction_id|user_id|     amount|transaction_type|merchant_category|country|hour|device_risk_score|ip_risk_score|is_fraud|
+--------------+-------+-----------+----------------+-----------------+-------+----+-----------------+-------------+--------+
|          9608|    363|4922.587542|             ATM|           Travel|     TR|  12|      0.992346524|  0.947907677|       1|
|           456|    692|48.01830286|              QR|             Food|     US|  21|      0.168571239|  0.224057403|       0|
|          4747|    587|  136.88196|          Online|           Travel|     TR|  14|      0.296127249|  0.125057692|       0|
|          6934|    445|80.53471896|             POS|         Clothing|     TR|  23|      0.124800978|  0.159242505|       0|
|          1646|    729|120.0411582|          Online|          Grocery|     FR|  16|      0.098128954|  0.027541969|       0|
|          2183|    944|97.10862458|             POS|         Clothing|     DE|  17|      0.235399432|  0.105454051|       0|
|          1919|    829|166.2092619|          Online|           Travel|     UK|  12|      0.115905909|  0.223717843|       0|
|          3479|    845|96.51263663|          Online|          Grocery|     US|   7|      0.082250433|   0.03402301|       0|
|          6796|    129|83.33870063|              QR|             Food|     DE|  16|      0.021774441|  0.279597507|       0|
|          5129|    249|89.69573135|              QR|          Grocery|     UK|   6|      0.095353437|  0.136336413|       0|
|          7402|    701|101.4038267|              QR|           Travel|     FR|   7|      0.164793935|  0.291939625|       0|
|          6188|    116|106.5634436|              QR|             Food|     DE|  23|      0.136235969|  0.135830553|       0|
|          8538|    679|55.36360145|          Online|             Food|     US|  10|      0.074304264|  0.128760255|       0|
|          8410|    559|99.41209708|          Online|             Food|     DE|  10|      0.084773552|  0.014333689|       0|
|          8201|    564|94.54560514|             ATM|         Clothing|     TR|  20|      0.133489727|  0.162009834|       0|
|          2187|    336|133.4106627|             POS|             Food|     UK|  19|      0.083132164|  0.286020189|       0|
|          5666|    166|157.9981233|              QR|      Electronics|     FR|  19|      0.207533005|  0.019961657|       0|
|          1419|    288|202.9958954|              QR|          Grocery|     TR|  23|      0.156544299|  0.082757567|       0|
|          6484|    984|88.50981483|              QR|           Travel|     UK|  19|       0.14780303|  0.206480204|       0|
|          2480|    137|115.3065595|          Online|             Food|     FR|  14|      0.206384378|  0.068766353|       0|
+--------------+-------+-----------+----------------+-----------------+-------+----+-----------------+-------------+--------+
only showing top 20 rows

root
 |-- transaction_id: integer (nullable = true)
 |-- user_id: integer (nullable = true)
 |-- amount: double (nullable = true)
 |-- transaction_type: string (nullable = true)
 |-- merchant_category: string (nullable = true)
 |-- country: string (nullable = true)
 |-- hour: integer (nullable = true)
 |-- device_risk_score: double (nullable = true)
 |-- ip_risk_score: double (nullable = true)
 |-- is_fraud: integer (nullable = true)
```

6. Utilizando Spark SQL y MLlib

Una vez cargado el dataset en HDFS, hice una exploración sencilla utilizando Spark SQL para conocer la cantidad de transacciones fraudulentas identificadas en cada uno de los diferentes tipos de transacción y también otra exploración para conocer la totalidad de transacciones y el porcentaje que representan los fraudes.

transaction_type	total_trxn	total_frauds	percentage_fraud
ATM	493	29	5.88235294117647
Online	489	25	5.11247443762781
QR	481	22	4.57380457380457
POS	537	20	3.72439478584730

Mediante un análisis sencillo del dataset, pretendo predecir las transacciones fraudulentas de un conjunto de prueba en un modelo de regresión logística entrenado con el 80% de los registros del dataset, dando como resultado de los primeros 20 registros, que ninguno resultó ser fraude.

Fue requerido utilizar la estrategia de One-Hot encoding para el tipo de transacción para aplicar el modelo de regresión logística. La precisión del modelo fue de 82.78% por lo que se considera una buena aproximación.

transaction_type	amount	is_fraud	probability	prediction
POS	133.631288	0	[0.98835716102076...	0.0
QR	1.0	0	[0.99248317683488...	0.0
QR	31.2387441	0	[0.99067538703514...	0.0
POS	88.56324869	0	[0.99155142846704...	0.0
ATM	94.21517585	0	[0.98334566538553...	0.0
POS	135.3857441	0	[0.98821115789106...	0.0
QR	150.672288	0	[0.97827477454637...	0.0
ATM	148.3972757	0	[0.97560937526957...	0.0
Online	65.40343388	0	[0.98521161137799...	0.0
POS	149.7946708	0	[0.98694161757960...	0.0
QR	188.3025152	0	[0.97171838856427...	0.0
Online	126.7271807	0	[0.97720728327205...	0.0
Online	136.9411642	0	[0.97551359625661...	0.0
POS	135.1656531	0	[0.98822957229874...	0.0
ATM	101.6127115	0	[0.98245217000298...	0.0
Online	32.84373662	0	[0.98826102879081...	0.0
ATM	101.3627816	0	[0.98248311187917...	0.0
Online	131.1947972	0	[0.97648101502771...	0.0
QR	53.7890518	0	[0.98905267095104...	0.0
POS	117.9824579	0	[0.98958281627718...	0.0

only showing top 20 rows

7. Mejoras que se pueden aplicar

Toda vez que el objetivo de la actividad fue emular un sistema de datos distribuido mediante Docker, la gestión de los datos distribuidos mediante Spark y potenciar el análisis mediante MLlib, considero que se cumplió y que los siguientes pasos que pueden ayudar a mejorar el análisis de datos es aplicar una limpieza de datos para obtener una mejor calidad de datos, evaluar los VIF para eliminar aquellas variables redundantes y quedarse solo con las que aporten valor al modelo, evaluar multicolinealidad, heterocedasticidad, realizar EDA con gráficas para determinar si se requiere alguna transformación para llevar la distribución a una normal, además de aplicar el One-Hot Encoding para las demás variables independientes con la intensión de incluirlas en el modelo de regresión logística y mejorar el coeficiente de determinación así como evaluar la precisión del modelo resultante.

Referencias

1. *Umitka. (s.f.). Synthetic Financial Fraud Dataset [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/umitka/synthetic-financial-fraud-dataset*
2. *UNIR. (s.f.). Ingeniería para el Procesado Masivo de Datos [Material de clase]. Maestría en Análisis y Visualización de Datos Masivos, Universidad Internacional de La Rioja (UNIR).*
3. *Muñoz Alvarado, M. A. (12 de junio de 2026). Guía paso a paso práctica 1 [Imagen]. Material de clase de Ingeniería para el Procesado Masivo de Datos, Maestría en Análisis y Visualización de Datos Masivos, Universidad Internacional de La Rioja (UNIR).*